Mathlib PR candidates from the AXLE-verified corpus

Riemann Labs (primaryhosting)

2026-08-11

Contents

1 Mathlib PR candidates from the AXLE-verified corpus

Date: 2026-08-11 **Author:** automated hunt over `aristotle/best_proofs/*.lean` (AXLE-verified subset) **Verification env:** lean-4.32.0 (AXLE `axle_verify.json`, `"verified": true`) **Novelty tooling actually available this run:** Loogle (`loogle.lean-lang.org`, JSON API — **working**). LeanSearch (`leansearch.net`) was **down** (HTTP 521) the entire run. **No local Mathlib checkout** was present (`/Users/acutis/Projects/zeta-23-lean/.lake/packages/mathlib` does not exist), so I could not `grep` Mathlib source directly. Novelty is therefore assessed by Loogle constant/type queries plus knowledge of Mathlib's naming — good signal, but **not** a substitute for building against the pinned Mathlib rev. Every "gap" below is marked **PLAUSIBLE-GAP (needs build-confirmation)**, never asserted as fact.

1.1 TL;DR (honest)

I found **one genuinely strong gap**, with a natural companion lemma that is also a gap:

- **`odd_sigma_one_iff`** — *the sum of divisors of an odd `n` is odd iff `n` is a perfect square* — and its prerequisite **`isSquare_iff_even_factorization`** — *`n` ($ $0) is a square iff every exponent in its prime factorization is even.*

Both come from a single AXLE-verified file, both return **0 Loogle hits**, and the square lemma sits exactly parallel to an existing Mathlib lemma (`Nat.squarefree_iff_factorization_le_one`) whose *square* analogue is simply missing. That parallel is the single most convincing piece of evidence that this is a real hole, not a search miss.

Everything else I looked at is either **already in Mathlib** (Cassini, Cauchy–Davenport, Ruzsa triangle inequality, Plünnecke–Ruzsa) or a **niche fixed-n computation** (Redheffer 3–6, ZMod-5 Cauchy–Davenport instance, Schur `S(2)<5`) that is not itself a general theorem worth upstreaming.

1.2 1. Shortlist

1.2.1 Confirmed ALREADY IN MATHLIB — NOT PR-worthy

These verified proofs are thin wrappers that *call* the Mathlib lemma they "prove". Verifying them at Lean 4.32 is not a contribution.

Our theorem	File	Why it's already in Mathlib
`AdditiveComb.plunnecke_ruzsa_add_two` (Ruzsa triangle ineq, `|A-C||B| $\leq$ |A-B||B-C|`)	`AdditiveComb_plunnecke_ruzsa_add_two.lean`	Proof body is literally `Finset.ruzsa_triangle_inequality_sub_su` + a symmetry rewrite.
`AdditiveComb.freiman_two_A` (2|A|-1 $\leq$ |A+A|)	`AdditiveComb_freiman_two_A.lean`	One-liner over `cauchy_davenport_add_of_linearOrder_isC`
`AdditiveComb.sumset_lower_bound` (|A|+|B|-1 $\leq$ |A+B| over $\mathbb{Z}$)	`AdditiveComb_sumset_lower_bound.lean`	Re-proof of the ordered Cauchy–Davenport bound, which Mathlib already has (`cauchy_davenport_add_of_linearOrder_is`
`Brockian.Cassini.cassini` (Fibonacci Cassini identity)	`Brockian_Cassini_cassini.lean`	Standard; Mathlib has `Nat.fib` Cassini-type identities / `Nat.fib_add_two` machinery. Classic, not novel.

1.2.2 Niche fixed-n COMPUTATIONS — not general theorems, NOT PR-worthy

Real math, but each proves a single numeric instance by `decide`/case analysis, not a theorem Mathlib wants.

Our theorem	File	Assessment
`Riemann.Redheffer.det_eq_mertens_3_4_5_6` (`det R = M(n)` for n=3..6)	`Riemann_Redheffer_det_eq_mertens_3_4_5_6.lean`	**niche-computation.** The *general* Redheffer identity `det R = M(n)` is genuinely NOT in Mathlib and *would* be a great contribution — but our files only do fixed 3$×3...6×$6 matrices by `decide`. They do not prove the general theorem, so they are not upstreamable as-is. (Flagged as a future target, see §3.)
`AdditiveComb.cauchy_davenport_Z5` (a `{0,1}+{0,2}` instance in `ZMod 5`)	`AdditiveComb_cauchy_davenport_Z5.lean`	Niche-computation (`decide`). Cauchy–Davenport itself is in Mathlib.

Our theorem	File	Assessment
`AdditiveComb.schur_five` (`S(2) < 5`, 2-colouring of {1..5})	`AdditiveComb_schur_five.lean`	niche-computation (`decide`).

1.2.3 PLAUSIBLE-GAP candidates (general lemmas, not in Mathlib per Loogle)

All from `Brockian_BetrothedNumbers_coprime_sameParity_twentyOne_primeFactors.lean` and `Brockian_BetrothedNumbers_coprime_pair_four_primeFactors.lean` (both AXLE `"verified": true`). The *betrothed-number* results themselves are too bespoke for Mathlib, but these files incidentally prove several **domain-independent** number-theory lemmas.

#	Lemma	Exact statement (from best_proofs)	Loogle	Assessment
A	`isSquare_iff_even_factorization`	`{n : $\mathbb{N}$} (hn : n $\neq$ 0) : IsSquare n ↔ ∀ p ∈ n.primeFactors, Even (n.factorization p)`	`IsSquare+Nat.factorization` = **0**; `Even+Nat.factorization` = **0**; name `Nat.isSquare*` = **0**	**PLAUSIBLE-GAP (strong).** Direct parallel of the *existing* `Nat.squarefree_iff_factoriza` the square version is absent.
B	`odd_sigma_one_iff`	`{n : $\mathbb{N}$} (hn : n $\neq$ 0) (hodd : Odd n) : Odd ($\sigma$ 1 n) ↔ IsSquare n`	`Odd+ArithmeticFunction.sigma` = **0**; `+IsSquare` = **0**	**PLAUSIBLE-GAP (strong).** Classical named result ("σ(odd n) odd ⟺ n square"). Depends on A. **This is the flagship.**
C	`abundancy_le_prod_p_div_pm1_of_primeFactors`	`{N : $\mathbb{N}$} (hN : N $\neq$ 0) : ($\sigma$ 1 N : ℚ) / N ≤ ∏ p ∈ N.primeFactors, (p : ℚ)/(p-1)`	`ArithmeticFunction.sigma+Nat.primeFactors` returns only the *equality* `sigma_eq_prod_primeFactors_...`, not this bound	**PLAUSIBLE-GAP (moderate).** Generalizes rational abundancy bound $\sigma(N)/N \leq p/(p-1)$. Niche but real and clean.

#	Lemma	Exact statement (from best_proofs)	Loogle	Assessment
D	`sigma_mul_prod_pred_le`	`(M : ℕ) (hM : M ≠ 0) : σ 1 M ≤ ∏ p ∈ M.primeFactors, (p^2 - 1)`	same query as C: only the equality exists	**PLAUSIBLE-GAP (moderate).** ℕ-form of C. Slightly awkward statement (`p-1` truncated subtraction); C is the better-stated version.
E	`two_mul_sigma_one_le`	`(k : ℕ) : 2 * ArithmeticFunction.sigma 1 k ≤ k * (k + 1)`	`2 * sigma 1 _ ≤ _ * (_ + 1)` (pattern-match may be too strict)	**PLAUSIBLE-GAP (weak).** "$\sigma(k) \leq$ triangular(k)". Trivial to prove (divisors ≤ range), so may be considered too minor / already derivable; lower value.

Supporting general lemmas in the same files that are individually too small to PR but bolster B: `odd_prod_iff` (`Odd (∏ p ∈ s, f p) ↔ ∀ p ∈ s, Odd (f p)`) and `odd_geom_sum_iff` (`Odd (∑ k ∈ range m, p ^ k) ↔ Even m`, p odd).

1.3 2. Strongest genuine-gap candidate (the PR)

Recommended contribution: a small PR adding lemma A as a foundation and lemma B as the headline. B is the more interesting *named* theorem; A is its clean prerequisite and is independently useful, sitting exactly next to an existing Mathlib lemma. Contributing both together is the most defensible unit.

1.3.1 2a. Lemma A — square $\iff$ even factorization exponents

```
theorem Nat.isSquare_iff_even_factorization {n : ℕ} (hn : n ≠ 0) :
    IsSquare n ↔ ∀ p ∈ n.primeFactors, Even (n.factorization p) := by
  constructor
  · rintro ⟨r, rfl⟩ p _
    have hr : r ≠ 0 := by rintro rfl; simp at hn
    rw [Nat.factorization_mul hr hr]; simp
```

```
  $\cdot$ intro h
    refine   p $\in$ n.primeFactors, p ^ (n.factorization p / 2), ?_
    rw [← Finset.prod_mul_distrib]
    conv_lhs => rw [← Nat.factorization_prod_pow_eq_self hn]   -- see note
    refine Finset.prod_congr rfl (fun p hp => ?_)
    obtain  k, hk  := h p hp
    rw [hk, ← pow_add]; congr 1; omega
```

- **Provenance:** `Brockian_BetrothedNumbers_coprime_sameParity_twentyOne_primeFactors.lean` (AXLE verified, lean-4.32.0). The extracted proof used a `Nat.factorization_prod_pow_eq_self` rewrite in the reverse direction; the `conv_lhs` line above is the intended form (the raw file's `conv_lhs =>` block must be completed when de-namespaced — noted as an adaptation item).
- **Where it lives:** `Mathlib/Data/Nat/Factorization/Basic.lean` (or `.../Factorization/Defs.lean`), immediately beside `Nat.squarefree_iff_factorization_le_one`. Name it `Nat.isSquare_iff_even_fact` (drop the betrothed namespace).
- **Dependencies (all already in Mathlib):** `Nat.factorization_mul`, `Nat.factorization_prod_pow_eq_s` `Finset.prod_mul_distrib`, `Finset.prod_congr`, `IsSquare`. No new imports.
- **Strategy:** forward direction — a square `r*r` doubles every exponent; reverse — rebuild `n` as the square of `p^(e_p/2)` using that each `e_p` is even.

1.3.2 2b. Lemma B — sum of divisors of an odd number is odd $\iff$ it is a square (headline)

```
theorem ArithmeticFunction.odd_sigma_one_iff {n : $\mathbb{N}$} (hn : n $\neq$ 0) (hodd : Odd n
    Odd ($\sigma$ 1 n)  IsSquare n := by
  have hoddp :  p $\in$ n.primeFactors, p % 2 = 1 := fun p hp => by
    rcases (Nat.prime_of_mem_primeFactors hp).eq_two_or_odd with h2 | h1
    $\cdot$ exact absurd (h2  Nat.dvd_of_mem_primeFactors hp) (by
        simpa [Nat.odd_iff] using hodd) |>.elim  -- p=2 contradicts n odd; see note
    $\cdot$ exact h1
  have hfac : $\sigma$ 1 n
      =  p $\in$ n.primeFactors, $\sum$ k $\in$ Finset.range (n.factorization p + 1), p ^ k :=
    rw [ArithmeticFunction.sigma_one_apply, Nat.sum_divisors hn]
  rw [hfac, odd_prod_iff, Nat.isSquare_iff_even_factorization hn]
  exact forall _congr fun p hp => odd_geom_sum_iff (hoddp p hp)
```

with the two one-line helpers (also from the file):

```
theorem odd_prod_iff (s : Finset $\mathbb{N}$) (f : $\mathbb{N}$ $\to$ $\mathbb{N}$) :
    Odd (  p $\in$ s, f p)    p $\in$ s, Odd (f p) := by
  simp only [← Nat.not_even_iff_odd, even_iff_two_dvd,
    Prime.dvd_finset_prod_iff Nat.prime_two.prime]
  push_neg; rfl

theorem odd_geom_sum_iff {p m : $\mathbb{N}$} (hp : p % 2 = 1) :
    Odd ($\sum$ k $\in$ Finset.range (m + 1), p ^ k)  Even m := by
  have h : ($\sum$ k $\in$ Finset.range (m + 1), p ^ k) % 2 = (m + 1) % 2 := by
```

```
  rw [Finset.sum_nat_mod]; simp [Nat.pow_mod, hp]
rw [Nat.odd_iff, h, Nat.even_iff]; omega
```

- **Provenance:** same file, AXLE verified.
- **Where it lives:** `Mathlib/NumberTheory/Divisors.lean` or the `ArithmeticFunction.sigma` file (`Mathlib/NumberTheory/ArithmeticFunction/*`). `odd_prod_iff` is general enough to live in `Mathlib/Algebra/BigOperators/...` (it may already exist in some form — check before including).
- **Dependencies:** lemma A, `ArithmeticFunction.sigma_one_apply`, `Nat.sum_divisors`, `Nat.prime_of_mem_primeFactors`, `Nat.Prime.eq_two_or_odd`, `Finset.sum_nat_mod`. All in Mathlib.
- **Adaptation required (be honest):** the extracted proofs above are lightly hand-massaged from the raw file to read cleanly; the `p = 2` contradiction step and the `conv_lhs` in A must be re-closed against the *current* Mathlib API, which has drifted since the pinned 4.32 build (lemma renames, `simp` set changes). Treat the code as a faithful *strategy*, not a copy-paste PR.

1.3.3 Why B over the others

- **A** is safe but small; **C/D** are niche and C's statement (rational, no truncated subtraction) is the only clean form; **E** is arguably too trivial. **B** is a *named classical theorem* that is (i) confirmed absent by Loogle, (ii) genuinely useful (perfect-number / odd-perfect-number folklore uses it), and
  (iii) already fully proved and AXLE-verified in our corpus. It is the best story *and* the best evidence.

1.4 3. A second, higher-effort target worth flagging (not ready)

General Redheffer determinant identity `det R = M(n)` (Mertens function). Our corpus only proves n=3..6 by `decide`, but the *general* theorem is a well-known, genuinely-missing Mathlib result and a much bigger prize than lemma B. It is **not** a candidate now (we have no general proof), but if the goal is impact, this is the target to attempt a real proof of. Flagged for Chris, not recommended for this PR.

1.5 4. Honest caveats

1. **Verifying ≠ contributing.** A proof that AXLE marks `"verified": true` at lean-4.32.0 only means it type-checks *there*. If Mathlib already has the lemma (Cassini, Cauchy–Davenport, Ruzsa, Plünnecke), the file is a re-proof and worth **zero** as an upstream contribution. Most of our celebrated theorems (Fermat, Wilson, Cantor, quadratic reciprocity, IVT, …) are in exactly this bucket — **already in Mathlib, not novel.** This shortlist deliberately excludes them.
2. **Novelty here is Loogle-only.** LeanSearch was down and there is no local Mathlib to grep. Loogle constant/type queries are strong but can miss a lemma stated with different constants (e.g. a general `UniqueFactorizationMonoid/multiplicity` version of lemma A). **Before**

opening any PR, build against the pinned Mathlib rev and `exact?`/`rw?`/`leansearch` the target statements. If A turns out to exist in a `UFM`/`Associates` form, the *concrete `Nat.factorization` restatement* may still be accepted, but that is a judgement call for Mathlib reviewers.

3. **Mathlib review bar.** Even a true gap must meet Mathlib standards: correct namespace/name, docstring, golfed proof, no `decide` on anything sizeable, placement in the right file, and no duplication. The extracted proofs need de-namespacing (drop `Brockian.BetrothedNumbers.*`), API-drift fixes from 4.32 → current, and a maintainer's naming blessing.
4. **Nothing here was pushed, PR'd, or sent.** This is a review draft for Chris only.